

***Doubly Linked List – Destroy List
– Space Complexity and Algorithm***

Program :

```
void destroyList()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *current = head;
    Node *nextNode;
    while (current != nullptr)
    {
        nextNode = current->next;
        free(current);
        current = nextNode;
    }
    head = nullptr;
    cout << "List destroyed." << endl;
}
...
case 12:
    destroyList();
    break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra or temporary memory used by a function, not including the input data itself.

In the `destroyList` function, the new variables created are:

`Node *current`: A pointer to keep track of the node currently being processed. The memory for a pointer is constant.

`Node *nextNode`: A pointer to store the address of the next node before the current one is deleted. This also requires a constant amount of space.

The function's main purpose is to free memory, not allocate it. Since the amount of extra memory used is fixed (just two pointers), the auxiliary space complexity is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter-only Definition

This view considers only the space taken by the function's direct parameters. The function signature is `void destroyList()`.

The function takes no parameters.

Under this definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full-Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$. The while loop that iterates through the list affects the function's time complexity ($O(n)$), but not its space complexity.

Algorithm: Destroy List

DDESTROY(*START*)

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty."

EXIT.

[End of If structure.]

2. [INITIALIZE POINTERS]

a. Set PTR := START [PTR is a pointer to the current node being processed].

b. Declare NEXT_NODE [A pointer to hold the next node's address].

3. [TRAVERSE THE LIST AND FREE EACH NODE]

Repeat while PTR ≠ NULL:

a. Set NEXT_NODE := NEXTLINK[PTR] (Save the address of the next node before deleting the current one).

b. free(PTR) [Deallocate the memory of the current node].

c. Set PTR := NEXT_NODE [Move the pointer to the next node].

[END of Loop]

4. [RESET THE START POINTER]

Set START := NULL [Mark the list as empty].

5. [Output Statement That List Destroyed]

Write: "List Destroyed."

6. *Exit*
